

Overview

At **Sporty Group**, we power real-time sports betting experiences by ingesting data from various third-party **feed providers**. These feeds contain critical information such as **odds updates** and **event settlements** for live sports events.

Each provider delivers data in its own proprietary format. To make this data usable across our betting platform, we need to **normalize** these messages into a unified internal structure.

Your goal in this exercise is to build a **microservice** that acts as the **standardization layer** in our feed processing pipeline, transforming incoming feed data into a consistent internal format that can be used throughout the rest of our system.

Requirements

You must implement the solution using Spring Boot.

Your microservice will expose two HTTP POST endpoints, one for each of two providers that are part of this exercise.

Both ODDS_CHANGE and BET_SETTLEMENT messages will be sent to these endpoints.

The microservice must:

- Parse the incoming messages from each provider
- Convert them into a standardized internal format. This means that we need to have a standard ODDS_CHANGE message and standard BET_SETTLEMENT message.
- Send the standardized message to a message queue (mocked—no real queue implementation is required; logging or in-memory handling is sufficient). The format for these standardized messages is not provided. It is your job to define it, based on the data present in the exercise.

Delivery

- The solution needs to be 100% executable
- Provide a link to the GitHub repository where your solution is committed
 - Please make sure it is not private or restricted
- Provide a `README` file (documentation) how to run and use the solution

Message details and formats

There are two types of messages to handle:

- **ODDS_CHANGE**: Represents an update to the odds for a specific event.
- **BET_SETTLEMENT**: Represents the final result of markets for the event.

At the moment, these messages only apply to the **1X2** market, which is a standard in sports betting:

- **1**: Home team win
- **X**: Draw
- **2**: Away team win

Provider Message Formats

Each provider sends both message types to a single HTTP endpoint. The format differs per provider, and your task is to handle both formats and normalize them.

The formats for both of the providers are provided below. Each request will contain only one message—either a `BET_SETTLEMENT` or an `ODDS_CHANGE`. Requests will not contain multiple messages.

ProviderAlpha (Endpoint: </provider-alpha/feed>)

```
// ODDS_CHANGE message
{
  "msg_type": "odds_update",
  "event_id": "ev123",
  "values": {
    "1": 2.0,
    "X": 3.1,
    "2": 3.8
  }
}
// BET_SETTLEMENT message
{
  "msg_type": "settlement",
  "event_id": "ev123",
  "outcome": "1" // One of "1", "X", or "2"
}
```

ProviderBeta (Endpoint: </provider-beta/feed>)

```
// ODDS_CHANGE message
{
  "type": "ODDS",
  "event_id": "ev456",
  "odds": {
    "home": 1.95,
    "draw": 3.2,
    "away": 4.0
  }
}
// BET_SETTLEMENT message
{
  "type": "SETTLEMENT",
  "event_id": "ev456",
  "result": "away" // One of "home", "draw", or "away"
}
```